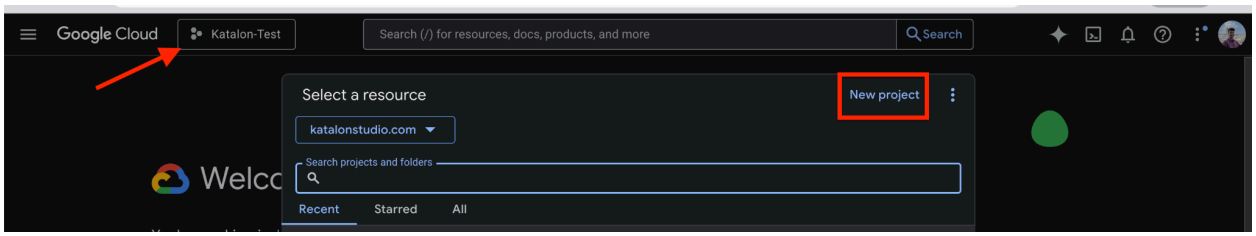
Data Driven Testing using Google Sheets in Katalon Studio

This guide provides a comprehensive, step-by-step process for integrating Google Sheets as a data source in **Katalon Studio v11 (running Java 21)**.

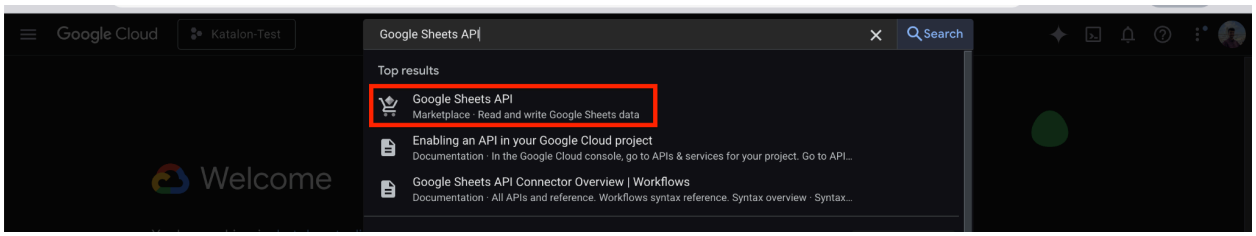
Phase 1: Google Cloud Console Setup

To allow Katalon to access your sheets, you must create an "Identity" in Google Cloud.

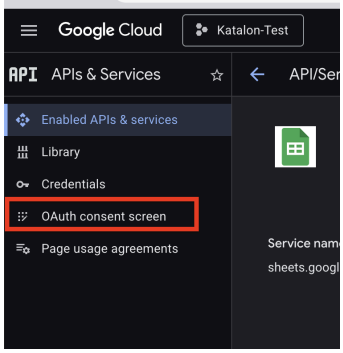
1. **Create a Project:** Go to the Google Cloud Console: [https://console.cloud.google.com/], create a **New Project**, and name it (e.g., `Katalon-Automation`).



2. **Enable API:** Search for **"Google Sheets API"** in the top search bar and click **Enable**.



3. **OAuth Consent Screen:** * Go to **APIs & Services > OAuth consent screen**.



   - Select **External**, fill in the **App Name** and **User Support Email**, then click **Save**.
   - Click **Publish App** on the dashboard to ensure the login token doesn't expire every 7 days.
4. **Create Credentials:** * Go to **APIs & Services > Credentials**.
   - Click **Create Credentials > OAuth client ID**.
   - Select **Application type: Desktop app**.
   - Download the resulting **JSON file**.
5. **Rename & Move:** Rename the downloaded file to exactly `client_secret.json` and move it into the **root folder** of your Katalon Project.

Phase 2: Add Java 21 Compatible Libraries

Since Katalon Studio v11 uses Java 21, you must manually add the modern Google API libraries (JAR files).

1. **Download the JARs:** Download the following 12 files from Maven Central:

You can download all the jars from this drive link:
[https://drive.google.com/drive/folders/14FPYzhZFVlrzaK7TpCJ72Ie144ELIVis?usp=drive_link]

   - `google-api-client-2.2.0.jar`
   - `google-oauth-client-1.34.1.jar`
   - `google-oauth-client-jetty-1.34.1.jar`
   - `google-oauth-client-java6-1.34.1.jar` (Required for the `AuthorizationCodeInstalledApp` class)
   - `google-http-client-1.43.3.jar`
   - `google-http-client-gson-1.43.3.jar`
   - `gson-2.10.1.jar`
   - `google-api-services-sheets-v4-rev20230815-2.0.0.jar`
   - `opencensus-api-0.31.1.jar`
   - `grpc-context-1.27.2.jar`
   - `opencensus-contrib-http-util-0.31.1.jar`
   - `Guava-32.1.3-jre.jar`

2. **Import to Katalon:** * In Katalon, go to **Project > Settings > Library Management**.
   - Click **Add**, select all 12 JARs, and click **Apply and Close**.

Phase 3: Create the Custom Keyword

This code acts as the bridge that reads the data.

1. **Create Package:** Under **Keywords**, create a new package named `com.google.utils`.
2. **Create Class:** Create a new Keyword named `ReadGoogleSheets`.
3. Use the code below:

```javascript
package com.google.utils



import com.google.api.client.auth.oauth2.Credential

import com.google.api.client.extensions.java6.auth.oauth2.AuthorizationCodeInstalledApp

import com.google.api.client.extensions.jetty.auth.oauth2.LocalServerReceiver

import com.google.api.client.googleapis.auth.oauth2.GoogleAuthorizationCodeFlow

import com.google.api.client.googleapis.auth.oauth2.GoogleClientSecrets

import com.google.api.client.googleapis.javanet.GoogleNetHttpTransport

import com.google.api.client.json.gson.GsonFactory  // UPDATED: Using Gson for Java 21

import com.google.api.client.util.store.FileDataStoreFactory

import com.google.api.services.sheets.v4.Sheets

import com.google.api.services.sheets.v4.SheetsScopes

import com.google.api.services.sheets.v4.model.ValueRange

import com.kms.katalon.core.annotation.Keyword

import com.kms.katalon.core.configuration.RunConfiguration
```

```java
import java.io.File

import java.io.FileInputStream

import java.io.FileNotFoundException

import java.io.InputStream

import java.io.InputStreamReader


public class ReadGoogleSheets {


	private static final String APPLICATION_NAME = "Katalon Google Sheets
Integration"

	private static final String TOKENS_DIRECTORY_PATH = "tokens"

	private static final String CREDENTIALS_FILE_PATH = "client_secret.json"


	// UPDATED: Use GsonFactory instead of JacksonFactory

	private static final GsonFactory JSON_FACTORY =
GsonFactory.getDefaultInstance()


	private static final List<String> SCOPES =
Collections.singletonList(SheetsScopes.SPREADSHEETS_READONLY)


	/**

	 * Main Keyword to fetch data from Google Sheet

	 */

	@Keyword
```

```
        public static List<List<Object>> getSpreadSheetRecords(String
spreadsheetId, String range) {

              try {

                     Sheets service = getSheetsService()



                     // Execute the Request

                     ValueRange response = service.spreadsheets().values()

                                   .get(spreadsheetId, range)

                                   .execute()



                     List<List<Object>> values = response.getValues()



                     if (values == null || values.isEmpty()) {

                            System.out.println("No data found.")

                            return []

                     } else {

                            return values

                     }

              } catch (Exception e) {

                     System.err.println("Error reading Google Sheet: " +
e.getMessage())

                     e.printStackTrace()

                     return []

              }
```

```
    }


    /**
     * Internal helper to handle Authorization
     */
    private static Sheets getSheetsService() throws IOException {
        String projectDir = RunConfiguration.getProjectDir()
        File credFile = new File(projectDir, CREDENTIALS_FILE_PATH)


        if (!credFile.exists()) {
            throw new FileNotFoundException("CRITICAL ERROR:
client_secret.json not found! Please check path: " +
credFile.getAbsolutePath())
        }


        InputStream keyStream = new FileInputStream(credFile)
        GoogleClientSecrets clientSecrets =
GoogleClientSecrets.load(JSON_FACTORY, new InputStreamReader(keyStream))


        // Set up the Flow (Login Logic)
        GoogleAuthorizationCodeFlow flow = new
GoogleAuthorizationCodeFlow.Builder(
                GoogleNetHttpTransport.newTrustedTransport(),
JSON_FACTORY, clientSecrets, SCOPES)
                .setDataStoreFactory(new FileDataStoreFactory(new
File(projectDir, TOKENS_DIRECTORY_PATH)))
```

```
                            .setAccessType("offline")

                            .build()


              // Trigger the User Auth (Browser Popup) on port 8888

              LocalServerReceiver receiver = new
LocalServerReceiver.Builder().setPort(8888).build()

              Credential credential = new AuthorizationCodeInstalledApp(flow,
receiver).authorize("user")



              return new
Sheets.Builder(GoogleNetHttpTransport.newTrustedTransport(), JSON_FACTORY,
credential)

                            .setApplicationName(APPLICATION_NAME)

                            .build()

       }

}
```

Phase 4: Prepare the Spreadsheet

1. **Get Sheet ID:** Open your Google Sheet. Copy the long string in the URL between `/d/` and `/edit`.
2. **Share Permissions:** Ensure the Google account you intend to use for authentication has at least **Viewer** access to that sheet.
3. **Identify Range:** Note the tab name (e.g., `Sheet1`) and the cells (e.g., `A2:B`).

Phase 5: Execute the Test

1. **The Test Case:** Create a new Test Case and call the keyword:
   Groovy

```
JavaScript

String sheetID = "YOUR_COPIED_ID"
String range = "Sheet1!A2:B"
List<List<Object>> data =
CustomKeywords.'com.google.utils.ReadGoogleSheets.getSpreadSheetR
ecords'(sheetID, range)
```

2. **First-Run Authentication:** * When you click **Run**, a browser window will open.
   - Select your Google account and click **Allow**.
   - If you see an "Unverified App" warning, click **Advanced > Go to [App Name] (unsafe)**.

3. **Verify Result:** Check the Katalon **Console**. Your spreadsheet rows will print as a List of Lists, ready for your automation loops.
4. Example Test Case:

```javascript
import com.google.utils.ReadGoogleSheets as ReadGoogleSheets
// 1. PASTE YOUR COPIED ID HERE
// 2. DEFINE YOUR RANGE (Sheet Name ! Start : End)
// 3. FETCH & PRINT
import com.kms.katalon.core.webui.keyword.WebUiBuiltInKeywords as WebUI
import static com.kms.katalon.core.testobject.ObjectRepository.findTestObject
String sheetID = '106gLU0uleZHHY-qIu4wi6I--YfYq4K9I5tBn34_suXU'
String range = 'Sheet1!A2:B'
List<List> data = ReadGoogleSheets.getSpreadSheetRecords(sheetID, range)
for (def row : data) {
   String user = row.get(0)
   String pass = row.get(1)
   println((('User: ' + user) + ' | Pass: ') + pass)
}


//Test case implementation
for (def row : data) {
   String user = row.get(0)
   String pass = row.get(1)
   WebUI.openBrowser('')
   WebUI.navigateToUrl('https://katalon-demo-cura.herokuapp.com')
   WebUI.click(findTestObject('Object Repository/Page_CURA Healthcare
Service/a_btn-make-appointment'))
   // Using the data from Google Sheets
   WebUI.setText(findTestObject('Page_CURA Healthcare Service/input_Username'),
user)
   WebUI.setText(findTestObject('Page_CURA Healthcare Service/input_Password'),
pass)
   WebUI.click(findTestObject('Page_CURA Healthcare Service/button_btn-login'))
   // Add validation or logout here
   WebUI.closeBrowser()
}
```